EDURA - Learning Management System

Software Requirements Specification (SRS) - Final Version

By

W.G.N.DANANJAYA
SE/2021/047
R.W.V.I.D.RAJAPAKSHA
SE/2021/029
P.D.D.RANSIKA
SE/2021/034
H.T.MADUSHANKA
SE/2021/011

A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc. SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

VERSION HISTORY

Version	Date	Author(s)	Changes
1.0	18/03/2026	Nadun Dananjaya, Imansha Dilshan, Danidu Ransika, Thushan Madhusanka	Initial SRS document creation
2.0	29/10/2026	Nadun Dananjaya, Imansha Dilshan, Danidu Ransika, Thushan Madhusanka	Final SRS. Requirements updated to describe the system as implemented: implementation status added to the functional and non-functional requirements, design and technology changes recorded (Chapter 21), and use cases, interfaces, data requirements, alternatives, feasibility and risks revised.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION 1

1.1 Project Background 1

1.2 System Overview 1

CHAPTER 2 3

PURPOSE 3

2.1 Document Purpose 3

2.2 Intended Audience 3

CHAPTER 3 4

SCOPE 4

3.1 System Name 4

3.2 System Capabilities 4

3.3 Explicit Exclusions 5

CHAPTER 4 7

DEFINITIONS, ACRONYMS AND ABBREVIATIONS 7

CHAPTER 5 8

DOCUMENT OVERVIEW 8

CHAPTER 6 9

PROJECT SCOPE 9

6.1 Business Objectives 9

6.2 Project Deliverables 9

6.3 Development Boundaries and Responsibilities 10

CHAPTER 7 11

STAKEHOLDERS 11

7.1 Client and System Owner 11

7.2 End Users 11

7.3 Development Team 11

7.4 Academic Evaluators 12

CHAPTER 8 13

FUNCTIONAL REQUIREMENTS 13

CHAPTER 9 18

NON-FUNCTIONAL REQUIREMENTS 18

CHAPTER 10 25
BUSINESS RULES 25
CHAPTER 11 27
CONSTRAINTS 27
11.1 Technical Constraints 27
11.2 Operational Constraints 28
11.3 Time and Resource Constraints 28
ASSUMPTIONS 30
12.1 User Capabilities and Access 30
12.2 External Systems and Dependencies 30
12.3 Operational and Business Assumptions 31
CHAPTER 13 32
USE CASE DIAGRAMS 32
13.1 System Actors 32
13.2 Student Subsystem Use Case Diagram 32
13.3 Teacher Subsystem Use Case Diagram 33
13.4 Administrator Subsystem Use Case Diagram 34
CHAPTER 14 36
USE CASE DESCRIPTIONS 36
14.1 UC-001: User Authentication and Login 36
14.2 UC-002: Upload Subscription Receipt 37
14.3 UC-003: Create Educational Module 39
14.4 UC-004: Attempt Timed Assessment 40
14.5 UC-005: Automated Subscription Payment via PayHere 41
CHAPTER 15 44
ACTIVITY DIAGRAMS 44
15.1 User Authentication and Verification Flow 44
15.2 Lesson and Content Management Flow 45
15.3 Assessment and Auto-Grading Flow 46
15.4 Subscription Payment Verification Flow 47
CHAPTER 16 49
EXTERNAL INTERFACE REQUIREMENTS 49
16.1 User Interfaces 49

16.2 Hardware Interfaces 49
16.3 Software Interfaces 50
16.4 Communication Interfaces 51
CHAPTER 17 53
DATA REQUIREMENTS 53
17.1 Logical Data Entities 53
17.2 Data Storage Mechanisms 54
17.3 Data Security and Integrity 54
17.4 Data Retention and Backup 55
CHAPTER 18 56
ALTERNATIVE SOLUTIONS CONSIDERED 56
18.1 Architectural Approach: Monolithic vs. Decoupled RESTful API 56
18.2 Platform Delivery: Native Mobile Application vs. Responsive Web Application 56
18.3 Payment Processing: Automated Gateway vs. Manual Verification vs. Hybrid Approach 57
18.4 Content Delivery: Local Server Storage vs. Third-Party Media Platforms 57
18.5 Database: MongoDB vs. PostgreSQL 58
18.6 Email Delivery: SMTP Server vs. Transactional Email API 58
18.7 Inter-Service Communication: Synchronous REST vs. Asynchronous Events 58
CHAPTER 19 59
FEASIBILITY STUDY 59
19.1 Technical Feasibility 59
19.2 Economic Feasibility 60
19.3 Operational Feasibility 60
19.4 Schedule Feasibility 61
CHAPTER 20 62
RISK ANALYSIS 62
20.1 Identified Risks and Mitigation Strategies 62
20.2 Continuous Risk Monitoring 66
CHAPTER 21 67
CHANGES FROM VERSION 1.0 67
CHAPTER 22 69

REFERENCES 69
CLIENT REQUIREMENT SIGN-OFF 72

LIST OF TABLES

Table 1:Definitions, Acronyms and Abbreviations 7
Table 2: Functional Requirements 13
Table 3: Non-Functional Requirements 18
Table 4: UC-001 – User Authentication and Login 36
Table 5: UC-002 – Upload Subscription Receipt 37
Table 6: UC-003 – Create Educational Module 39
Table 7: UC-004 – Attempt Timed Assessment.....40
Table 8: UC-005 – Automated Subscription Payment via PayHere 41
Table 9: Risk Analysis and Mitigation Strategies 62

LIST OF FIGURES

Figure 1: Student Subsystem Use Case Diagram 33
Figure 2: Teacher Subsystem Use Case Diagram.....34
Figure 3: Administrator Subsystem Use Case Diagram.....35
Figure 4: User Authentication and Verification Activity Diagram 45
Figure 5: Lesson and Content Management Activity Diagram.....46
Figure 6: Assessment and Auto-Grading Activity Diagram.....47
Figure 7: Subscription Payment Verification Activity Diagram.....48

CHAPTER 1

INTRODUCTION

1.1 Project Background

The transition of quality education toward online platforms has been increasingly observed; however, traditional learning environments remain frequently constrained by limited resources and difficulties in continuous progress tracking. Within the operational context of the ICT - Dimuthu educational institute, it has been identified that the majority of administrative and educational tasks are executed manually. Consequently, this manual execution leads to operational delays and procedural errors. Furthermore, the evaluation of student performance is recognized as a time-consuming and inconsistent process under the current framework.

The preparation of individual student progress reports presents significant administrative difficulties for educators. The absence of a centralized platform for managing lessons, quizzes, and student data results in limited tracking and monitoring capabilities. The mitigation of such administrative burdens and error costs through the adoption of new learning technologies, as opposed to traditional manual tracking methods, has been strongly supported by recent educational research [1][2]. Studies have demonstrated that LMS platforms can significantly reduce administrative workload while improving the consistency and accuracy of educational assessments and progress tracking [3]. Therefore, the necessity for a comprehensive digital solution was established to overcome these operational bottlenecks and optimize the institute's educational delivery.

1.2 System Overview

To address the aforementioned limitations, the EDURA Learning Management System is proposed as a comprehensive educational platform. The system is specifically designed to enable educators, students, and administrators to efficiently manage courses, digital lessons, and interactive assessments. Through the implementation of this centralized platform, the educational process is transformed to be highly flexible, engaging, and measurable.

Role-based access control is utilized to ensure secure interaction across different user categories (Admin, Teacher, and Student). Furthermore, core functionalities such as structured lesson management with video integration, automated quiz grading, and comprehensive progress tracking are incorporated into the system architecture. The capability of Learning Management Systems to utilize data from student interactions for

early interventions and improved learning outcomes has been highlighted as a critical advantage in recent academic studies [4][5].

In alignment with these findings, the learning experience within EDURA is further enhanced by interactive gamification elements, including performance-based leaderboards and achievement badges. Finally, measurable outcomes are provided to stakeholders through PDF progress reports that are generated and emailed to students, and course-completion eligibility is evaluated by the system. The generation of the certificate document itself and the Excel export of reports were not delivered in the final release (see Chapter 21).

CHAPTER 2

PURPOSE

2.1 Document Purpose

A comprehensive description of the EDURA Learning Management System is provided by this Software Requirement Specification (SRS) document. This final version (2.0) describes the system as it was implemented and demonstrated: the requirements of version 1.0 are retained, the implementation status of each requirement is recorded, and every change made during development is summarised in Chapter 21. The functional and non-functional requirements, system constraints, and design specifications necessary for the deployment and operation of the EDURA platform are explicitly outlined herein. Furthermore, the contractual basis for the project is established by this document, ensuring that a clear and mutual understanding of system capabilities, limitations, and expectations is maintained by all involved parties.

2.2 Intended Audience

This document is intended to be utilized by various project stakeholders to guide the development, testing, and validation processes. Specifically, the SRS is directed toward the development team (Team ZYNAPTRIX), the primary client (Mr. Dimuthu Prabath), other relevant project stakeholders, and system administrators. It is intended that the document will be used by the development team as a foundational blueprint for system engineering, while it will be referenced by the client and administrators to verify that the delivered system aligns comprehensively with the agreed-upon requirements.

CHAPTER 3

SCOPE

3.1 System Name

The system is officially designated as EDURA - Learning Management System.

3.2 System Capabilities

EDURA is defined as a comprehensive web-based learning management system that is specifically tailored for the "ICT - Dimuthu" educational institute to facilitate online education for Advanced Level (A/L) students aged 15-18 years. The creation, management, and delivery of digital courses, assessments, and learning materials by educators will be enabled by the system. Furthermore, an engaging, interactive learning experience that is enhanced by gamification elements will be provided to the students.

The specific functionalities to be executed by the system are outlined as follows:

- Role-based access control will be provided for administrators, teachers (content creators/editors), and students.
- Comprehensive course and lesson management is enabled, incorporating private YouTube video content, Cloudinary-hosted notes and organised learning modules (course, module and lesson hierarchy with a draft, published and archived lifecycle). Data is stored in PostgreSQL (Neon), with one database per service, and Redis is used for one-time passwords, sessions, timed assessment sessions, leaderboards and report caching.
- Multiple-choice and true/false assessments are supported with automatic grading; short-answer questions are supported and are held for manual review by the teaching staff.
- Student progress, encompassing video viewing completion, quiz performance, and overall course advancement, will be continuously tracked.
- Progress reports for individual students are generated as PDF documents and emailed to the student; summary analytics are available to administrators. Excel export and class-level report export were not delivered.

- Secure authentication (email and password with bcrypt hashing, JWT access and refresh tokens, and optional Google sign-in) is paired with email verification. Anti-cheating monitoring of assessments detects and records tab-switching and focus-loss events; a full browser lockdown is not enforced.
- Gamification features are incorporated through performance-based leaderboards (Redis sorted sets) and achievement badges. Research has demonstrated that such gamification elements significantly enhance student engagement, motivation, and academic performance in online learning environments [6][7][8].
- Course-completion eligibility is evaluated automatically and a certificate record is created when a student completes a course; the generation of a downloadable certificate document and a certificate page in the student interface were not delivered.
- The AI-powered learning assistant (Google Gemini) planned in version 1.0 was not implemented and is recorded as future work. Learner support is provided through a support-chat channel to the administration team.
- Subscription payment processing is facilitated through a hybrid design. Bank-deposit receipt uploads with administrative verification are the active payment method. Automated activation through the PayHere payment gateway (checkout request, authenticated webhook and idempotent processing) is implemented in the backend against PayHere's sandbox, but the PayHere option is hidden in the checkout page of the delivered release.
- Additional capabilities implemented during development include class packages, subject streams and grade subjects, video checkpoint questions, feedback collection, an administrator support chat, an audit log, and a student referral and wallet ledger.

3.3 Explicit Exclusions

The boundaries of the project are defined by the following explicit exclusions:

- Single sign-on integration with external identity systems is not supported. Optional Google sign-in is provided alongside email-and-password authentication; no other social login is provided.
- Live video streaming and real-time virtual classroom functionalities are not included.

- Integration with external learning management systems or third-party educational platforms is not supported.
- Mobile native applications (iOS/Android) are not part of the initial scope; the system will be strictly restricted to a web-responsive application.
- Advanced AI features, such as automated content generation or personalized learning path recommendations, are excluded from this release and reserved for future development phases.

CHAPTER 4

DEFINITIONS, ACRONYMS AND ABBREVIATIONS

All domain-specific terms, acronyms, and abbreviations utilized throughout this Software Requirement Specification document are defined in the following table to ensure clarity and a mutual understanding among all stakeholders.

Table 1:Definitions, Acronyms and Abbreviations

Term / Acronym	**Definition**
AI	Artificial Intelligence - Technology by which human intelligence is simulated by machines.
AKS	Azure Kubernetes Service - A managed Kubernetes platform used to run the containerised services.
API	Application Programming Interface - A set of protocols and tools utilized for the building of software applications.
CDN	Content Delivery Network - A geographically distributed network of servers employed to deliver content efficiently.
CI/CD	Continuous Integration and Continuous Delivery - Automated build, test and deployment pipelines (GitHub Actions).
JWT	JSON Web Token - A signed token used to carry the identity and role of an authenticated user.
LMS	Learning Management System - A software application utilized for the administration, documentation, tracking, reporting, and delivery of educational courses or training programs.
MCQ	Multiple Choice Question - A specific format of assessment utilized within the platform.
ORM	Object-Relational Mapping - A technique by which database tables are accessed as program objects (SQLAlchemy).
OTP	One-Time Password - A password that is generated to be valid for only a single login session or transaction.
PostgreSQL	An open-source relational database system, hosted on Neon, used for persistent storage.

RabbitMQ	A message broker used to exchange asynchronous events between the microservices.
REST	Representational State Transfer - An architectural style utilized for the design of networked applications.
SLA	Service Level Agreement - A formalized commitment established between a service provider and a client by which the expected level of service is defined.
SRS	Software Requirement Specification - The primary documentation artifact utilized to detail system requirements.
UI	User Interface - The visual elements through which the system is interacted with by users.
UX	User Experience - The overall experience of an individual by whom the system is used.

CHAPTER 5

DOCUMENT OVERVIEW

How the remainder of this Software Requirement Specification document is organized is outlined in this section. Following the initial introductory and definitional sections (Sections 1 through 5), the structural layout of the document is established as follows:

- The overall project scope and all relevant stakeholders are detailed in Sections 6 and 7, respectively.
- The core capabilities and quality attributes of the system are comprehensively documented as functional and non-functional requirements in Sections 8 and 9.
- The business rules, system constraints, and underlying assumptions by which the system behavior and development process are governed are defined in Sections 10, 11, and 12.
- System functionalities and user interactions are visually and textually illustrated through use case diagrams, detailed use case descriptions, and activity diagrams, which are provided in Sections 13, 14, and 15.
- The external interface expectations, including user, hardware, software, and communication interfaces, are specified in Section 16.

- Data requirements and database specifications are detailed in Section 17.
- Alternative solutions that were evaluated and the justification for the chosen approach are discussed in Section 18.
- A comprehensive feasibility study, encompassing technical, economic, operational, and schedule aspects, is presented in Section 19.
- Potential project risks and their corresponding mitigation strategies are analyzed in Section 20.
- Finally, the changes made to the requirements since version 1.0 are summarised in Section 21, and the external documents, standards, and sources cited throughout the document are listed in Section 22, which is subsequently followed by the formal client and project supervisor sign-off pages.

CHAPTER 6

PROJECT SCOPE

6.1 Business Objectives

The primary objective of the EDURA project is to establish the comprehensive digitalization of the educational processes currently utilized by the "ICT - Dimuthu" institute. The manual administrative and grading workloads currently experienced by the institute's teaching staff are intended to be significantly reduced through this transition. Furthermore, the accessibility and engagement of Advanced Level (A/L) ICT educational resources are to be enhanced for students through a centralized, highly available web-based platform.

6.2 Project Deliverables

The successful execution and delivery of a fully functional web-based Learning Management System is encompassed within the boundaries of this project. The specific artifacts and components to be delivered to the client by the development team (Team ZYNAPTRIX) are formally defined as follows:

- This final Software Requirement Specification (SRS) document, which records the requirements and the system as implemented.
- The complete compiled source code of the EDURA web application.

- A fully designed and configured database architecture utilized for the secure management of user profiles, course content, and assessment records.
- System documentation, including administrative and user manuals, by which the operation of the platform is guided.
- The deployment pipeline and a staging deployment of the system on Microsoft Azure Kubernetes Service; the production deployment is released through a pipeline with a manual approval gate.

6.3 Development Boundaries and Responsibilities

The development phase of the project is strictly constrained to the design, implementation, and deployment of the software functionalities explicitly detailed within this SRS document. The procurement or provision of physical hardware, network infrastructure, or end-user devices (such as computers or internet connections for the students) is not encompassed by the responsibilities of the project team.

Furthermore, the creation, curation, and uploading of the actual educational content (including video lectures, quiz questions, and study notes) are designated as the sole responsibility of the client (Mr. Dimuthu Prabath) and the appointed teaching staff. Only the digital platform through which this content is hosted, managed, and delivered will be provided by the development team. Finally, the entire project lifecycle, from requirement elicitation to final deployment, is required to be completed within the academic timeline and evaluation schedule specified by the Software Engineering Teaching Unit of the University of Kelaniya.

CHAPTER 7

STAKEHOLDERS

Various individuals, groups, and entities with whom the EDURA Learning Management System interacts, is interacted with, impacted by, or developed for, are identified and categorized within this section. The specific roles and responsibilities of each stakeholder group are defined as follows:

7.1 Client and System Owner

The role of the primary client and system owner is held by Mr. Dimuthu Prabath, the owner and principal lecturer of the "ICT - Dimuthu" educational institute. The overall business requirements, educational content, and final approval of the delivered system are provided by the client. The administrative and strategic oversight of the deployed platform will ultimately be maintained by this stakeholder.

7.2 End Users

The system is expected to be utilized directly by three primary categories of end-users:

- **Students:** The platform is intended to be accessed by Advanced Level (A/L) ICT students, primarily aged between 15 and 18 years. System features such as video lectures, quizzes, progress tracking, and gamified leaderboards will be engaged with by these users to achieve their educational objectives.
- **Teachers / Content Creators:** The creation, uploading, and management of educational materials (such as video modules and multiple-choice questions) are executed by the teaching staff. The progress of the students and the outcomes of automated and manual assessments will also be monitored by these users.
- **System Administrators:** The routine operational aspects of the platform are overseen by administrative staff. Platform configuration, user account management, and the manual verification of subscription payment receipts are handled by this user group.

7.3 Development Team

The architectural design, software development, quality assurance, and initial deployment of the system are designated as the explicit responsibilities of Team ZYNAPTRIX. The project requirements and technical deliverables outlined within this document are required

to be fulfilled by the team members (Nadun Dananjaya, Imansha Dilshan, Danindu Ransika, and Thushan Madhusanka).

7.4 Academic Evaluators

The academic oversight and final evaluation of the project are conducted by the Software Engineering Teaching Unit of the University of Kelaniya. The adherence of the project to formal software engineering standards, as well as the evaluation of the final system against academic grading criteria, is assessed by the appointed project supervisors and evaluation committees.

CHAPTER 8

FUNCTIONAL REQUIREMENTS

The specific behaviors, computations, and informational processing functions that are required to be performed by the EDURA Learning Management System are defined in this section. Each functional requirement has been assigned a unique identifier, categorized by priority (High, Medium, or Low), and traced back to its source stakeholder. The status column records the implementation status at the final release: Implemented, Partly implemented, or Not implemented. Requirements FR-016 to FR-020 were added during development.

Table 2: Functional Requirements

ID	Description	Priority	Source	Status
FR-001	Secure user authentication and authorization, utilizing role-based access control for Administrators, Teachers, and Students, shall be enforced by the system. Authentication uses email and password with JWT access tokens and refresh tokens.	High	Client	Implemented
FR-002	User registration and password recovery processes shall be validated through the implementation of email verification mechanisms. Registration is verified through a signed, time-limited link; password recovery and verification codes are one-time passwords with a 5-minute lifetime.	High	Client	Implemented
FR-003	The creation, structuring, and modification of lessons, including the embedding of private YouTube	High	Teachers	Implemented

	video links, the integration of Cloudinary media assets, and the uploading of supplementary notes, shall be facilitated for Teachers and Administrators.			
FR-004	The creation and configuration of interactive assessments, specifically encompassing Multiple-Choice Questions (MCQ), True/False formats, and short answer queries, shall be enabled. Short-answer questions are marked manually by the teacher.	High	Teachers	Implemented
FR-005	Submitted quizzes and objective assessments (MCQ and True/False) shall be automatically graded by the system immediately upon completion.	High	Client	Implemented
FR-006	As a fallback payment method, the uploading of manual payment receipts (bank deposit slips) by students shall be facilitated, which shall subsequently be verified and approved by system administrators. Receipts are stored in Cloudinary; this is the active payment method.	High	System Admin	Implemented
FR-007	For automated subscription processing, users shall be securely redirected to the PayHere checkout gateway by the system. The redirect is implemented against PayHere's sandbox and is hidden in the checkout page (LIM-10).	High	Client	Partly implemented

FR-008	Payment status webhooks and server-to-server callbacks dispatched by the PayHere gateway shall be securely received, authenticated, and processed by the system. The signature is verified (MD5, as PayHere specifies) and processing is idempotent on the transaction identifier; the flow was tested against a mock gateway only.	High	System Admin	Implemented
FR-009	Course access shall be automatically granted to the student by the system upon a successful payment status or administrator approval of a receipt, without further manual intervention. The grant is made by an enrollment event consumed by the Enrollment Service.	High	Students	Implemented
FR-010	Student progress, specifically measured by the percentage of videos watched, quiz scores achieved, and overall course completion status, shall be continuously tracked and recorded.	High	Client	Implemented
FR-011	Platform navigation and contextual learning support were to be provided through a Google Gemini AI-powered digital assistant. The assistant was descoped and is recorded as future work (LIM-01); learner support is provided through	High	Client	Not implemented

	a support chat to the administration team.			
FR-012	Analytics and performance reports for students should be generated and exportable. PDF progress reports are generated and emailed to the student; the Excel export and class-level export were not delivered (LIM-07).	Medium	Teachers	Partly implemented
FR-013	Anti-cheating measures shall be implemented during timed assessments to ensure academic integrity. Tab-switch and focus-loss events are detected, recorded and reported; browser lockdown is not enforced.	Medium	Client	Implemented
FR-014	Student engagement shall be stimulated through the integration of gamification elements, including achievement badges and performance-based leaderboards (Redis sorted sets).	Medium	Students / Client	Implemented
FR-015	Digital certificates of completion shall be automatically generated and issued to students upon the successful fulfillment of all course requirements. The eligibility rule, certificate record and certificate event are implemented; no certificate document is generated and the frontend has no certificate page (LIM-08).	Medium	Students	Partly implemented

FR-016	Class packages, subject streams and grade subjects shall be supported so that students can be enrolled in class-based groups of courses.	Medium	Client feedback / Team	Implemented
FR-017	Teachers shall be able to place checkpoint questions inside video lessons.	Medium	Client feedback / Team	Implemented
FR-018	Feedback collection and an administrator support chat shall be provided, together with an audit log of administrative actions.	Medium	Client feedback / Team	Implemented
FR-019	A student referral and wallet ledger shall be maintained.	Low	Team	Implemented
FR-020	Users shall optionally be able to sign in with Google; the account is linked to an existing account by email address.	Medium	Team	Implemented

Recent studies have validated the effectiveness of automated cheating detection systems in online assessments. Machine learning and deep learning approaches, including LSTM networks and computer vision techniques, have achieved detection accuracies exceeding 90% in identifying suspicious behaviors during virtual examinations [9][10][11]. The implemented mechanism is limited to behavioural monitoring (tab-switch and focus-loss detection with recorded violations); machine-learning-based and camera-based detection are not part of the delivered system.

CHAPTER 9

NON-FUNCTIONAL REQUIREMENTS

The qualitative attributes, performance metrics, and technical constraints by which the EDURA Learning Management System is strictly governed are outlined in this section. To ensure a high standard of software quality, each requirement has been categorized in alignment with the ISO/IEC 25010 software quality assurance framework. Furthermore, measurable acceptance criteria have been assigned to each non-functional requirement to facilitate precise system evaluation and testing.

Table 3: Non-Functional Requirements

ID	Category	Description	Measurable Metric / Acceptance Criteria	Priority
NFR-001	**Performance**	The system response time for standard page loads and query executions shall be minimized to ensure a seamless user experience.	95% of standard user interface interactions shall be processed and rendered in under 2.0 seconds under normal server load. **Final status:** Met for the endpoints tested on the staging environment: the 95th percentile was below 2.0 seconds (course list 1.8 seconds).	High
NFR-002	**Reliability**	A continuous and stable operational state shall be maintained by the system, ensuring that educational resources remain accessible to students.	A minimum system uptime of 99.9% (excluding scheduled maintenance windows) shall be maintained consistently on a monthly basis.	High

			Final status: Not measured. No uptime monitoring or service-level agreement was put in place; the staging environment runs on a low-cost cluster.	
NFR-003	**Reliability**	Automated data preservation mechanisms shall be implemented to prevent the loss of critical educational content and student records.	Full database backups shall be automatically executed every 24 hours. **Final status:** Not implemented by the project. No scheduled backup job exists; data protection depends on the managed database provider (Neon) and was not verified.	High
NFR-004	**Reliability**	Robust fault-tolerance mechanisms shall be implemented to manage connection timeouts or failed webhook deliveries from the external PayHere API, ensuring that payment statuses are accurately reconciled.	100% of missed or failed payment webhooks shall be detected by the system and queued for an automated status reconciliation polling process within 15 minutes of the initial transaction timeout. **Final status:** Not implemented. Payment webhooks	High

			are handled idempotently, but no reconciliation polling process exists; a missed webhook has to be resolved by an administrator (see NFR-004 in Chapter 21).	
NFR-005	**Usability**	The user interface shall be designed to be fully responsive, ensuring optimal readability and interaction across a wide variety of devices.	The platform shall seamlessly adapt to desktop, tablet, and mobile device screen resolutions without horizontal scrolling or broken UI elements. **Final status:** The interface is responsive; it has not been formally tested on small-screen devices (LIM-05).	High
NFR-006	**Security**	Sensitive user credentials, specifically passwords, shall be cryptographically secured prior to database storage.	100% of user passwords shall be encrypted utilizing the bcrypt hashing algorithm; plain text storage shall be strictly prohibited. **Final status:** Met. Passwords are stored as bcrypt hashes.	High

NFR-007	**Security**	Authentication verification requests shall be strictly time-bound to mitigate the risk of unauthorized account access.	One-Time Passwords (OTPs) generated for email verification shall be configured to expire exactly 5 minutes after issuance. **Final status:** Met. One-time passwords are held in Redis with a 300-second expiry.	High
NFR-008	**Security**	The storage of sensitive financial data, specifically credit or debit card numbers, upon the EDURA servers shall be strictly prohibited; all automated payment processing shall be delegated to and strictly governed by the security protocols of the PayHere gateway.	0% of primary account numbers (PAN) or CVV codes shall be transmitted to or stored within the EDURA database infrastructure. **Final status:** Met. Card details are entered only on PayHere's hosted page; no card data reaches EDURA.	High
NFR-009	**Scalability**	The server architecture and database connections shall be engineered to accommodate high volumes of simultaneous student access, particularly during assessment periods.	The system shall be capable of supporting a minimum of 500 concurrent user sessions without experiencing a performance degradation exceeding 20%.	Medium

			Final status: Not met. About 20 concurrent clients per service were supported on the low-cost staging cluster, and requests timed out at 40 (LIM-06, ADR-17).	
NFR-010	**Maintainability**	The backend architecture shall be structured in a systematic and decoupled manner to facilitate future updates and collaborative development.	The system shall be organized utilizing a modular REST API architecture, with 100% of API endpoints systematically documented. **Final status:** Partly verified. The backend consists of eleven FastAPI services with interactive API documentation generated from the code; completeness of the documentation was not audited.	Medium
NFR-011	**Maintainability**	The system shall be developed using industry-standard frameworks and languages to ensure long-term	The frontend shall be built using Next.js framework, backend services shall be implemented in Python, and all code shall follow PEP 8	High

		maintainability and developer accessibility.	(Python) and ESLint (JavaScript) style guidelines. **Final status:** Met, with tooling differences. The frontend uses Next.js with TypeScript and the backend uses Python with FastAPI; code style is enforced with Ruff and Black (Python) and ESLint (frontend) in the continuous integration pipeline.	
NFR-012	**Portability**	Platform accessibility across diverse operating environments shall be ensured by adhering to modern web standards.	Full system compatibility and uniform functionality shall be verified across the latest stable versions of Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge. **Final status:** Not fully verified. Automated browser tests run on Chromium only; Firefox, Safari and Edge were not formally tested.	Medium

CHAPTER 10

BUSINESS RULES

The fundamental operational policies, academic constraints, and access regulations by which the EDURA Learning Management System is governed are defined in this section. These rules must be strictly enforced by the system logic to ensure the integrity of the educational process and the financial operations of the institute.

- **BR-001 (Subscription Access):** Access to course materials and assessments shall be strictly granted to students under one of two conditions: either immediately and automatically upon the system's receipt of a successful transaction callback from the PayHere payment gateway, or upon the successful manual verification of an uploaded payment receipt by system administrators. If a subscription lapses without renewal via either authorized method, access to new course materials is automatically suspended by the system through a subscription-expired event. In the delivered release the bank-receipt route is the active one.
- **BR-002 (Assessment Grading):** Automated grading mechanisms shall only be applied to objective assessment formats, specifically Multiple-Choice Questions (MCQ) and True/False questions. Any subjective assessments must be flagged for manual review by the teaching staff.
- **BR-003 (Timed Examinations):** Timed quizzes and assessments must be completed within a single, continuous session. Upon the expiration of the allocated time limit, the ongoing assessment shall be automatically submitted by the system, regardless of completion status.
- **BR-004 (Certification Eligibility):** Digital certificates of completion shall only be issued to students upon the verified completion of all mandatory video modules (reaching 100% watch status) and the achievement of a predefined minimum passing score in all required course assessments. The eligibility rule and the certificate record are implemented; the certificate document is not generated.
- **BR-005 (Account Integrity):** One-Time Passwords (OTPs) generated for the purposes of account registration, email verification, or password recovery shall be rendered permanently invalid exactly five minutes after their initial generation.

- **BR-006 (Anti-Cheating Enforcement):** During the execution of timed, formal assessments, tab-switching and focus-loss detection is enforced and browser lockdown is not. Detected violations are recorded and reported to the system administrators and relevant teaching staff by email notification. The global online exam proctoring market, valued at $836.43 million in 2023 and projected to reach $1.99 billion by 2029, reflects the widespread adoption of automated integrity monitoring systems in educational institutions worldwide [13].

CHAPTER 11

CONSTRAINTS

The technical, operational, and environmental limitations by which the development, deployment, and ongoing operation of the EDURA Learning Management System are restricted are outlined in this section. These constraints must be strictly adhered to by the development team throughout the project lifecycle.

11.1 Technical Constraints

- **Platform Limitation:** The system is strictly constrained to a web-based architecture. The development of native mobile applications (iOS or Android) is explicitly excluded from the current project scope.
- **Connectivity Dependence:** Continuous and stable internet connectivity is required by all end-users (students, teachers, and administrators) to access the platform, view lessons, and submit assessments. Offline capabilities are not supported.
- **Browser Compatibility:** System compatibility is restricted to modern web browsers. Full functionality must be ensured only on the latest stable releases of Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.
- **Technology Stack Constraints:** The system frontend is built with the Next.js framework (App Router, TypeScript, Redux Toolkit). Backend microservices are implemented in Python with FastAPI to maintain consistency across service boundaries.
- **Database Architecture:** The system persists its data in PostgreSQL hosted on Neon, with one database per service, and uses Redis for one-time passwords, sessions, timed assessment sessions and leaderboards. MongoDB, which version 1.0 mandated, was replaced because relational integrity and transactional guarantees were needed (Chapter 21).
- **Cloud Platform Dependency:** The services are deployed on Microsoft Azure Kubernetes Service (staging) through GitHub Actions pipelines. The database (Neon), media delivery (Cloudinary) and email (Resend) are provided by external services. Migration to another cloud provider would require changes to the deployment manifests and pipelines.

- **Media Handling:** The platform is restricted to handling pre-recorded video uploads. Real-time virtual classrooms and live video streaming functionalities are not supported by the system architecture.
- **Authentication Limitations:** Authentication is based on the system's own email and password verification. Optional Google sign-in is supported; no other social login or Single Sign-On mechanism is provided.
- **Infrastructure Complexity:** The deployment environment relies on containerization (Docker) and orchestration (Kubernetes). Eleven independent services, an Nginx gateway, a Caddy HTTPS edge, a RabbitMQ broker and Redis must be deployed, networked and monitored, which is more complex than a monolithic application.

11.2 Operational Constraints

- **Manual Payment Processing:** The activation of student subscriptions is constrained by a manual verification process. Although the PayHere integration is implemented in the backend, it runs against PayHere's sandbox and is hidden in the checkout page, so receipt uploads must be visually verified and manually approved by system administrators before course access is granted.
- **Content Storage:** The capacity for video and lesson storage is inherently limited by the hosting and database infrastructure allocated during the deployment phase. Extensive scaling may require manual infrastructure upgrades by the client.
- **Third-Party Media Dependency:** The system's video delivery capabilities are strictly constrained by the terms of service, API rate limits, and bandwidth allocations imposed by YouTube and Cloudinary. Any policy changes or service degradations on these external platforms will directly impact the EDURA system.
- **Low-Cost Infrastructure:** The staging environment runs on a low-cost cluster to limit cloud expenditure. Performance testing showed that about 20 concurrent clients per service are supported and that requests time out at 40; the 500-user target of NFR-009 is therefore not met until the infrastructure is scaled.

11.3 Time and Resource Constraints

- **Academic Schedule:** The entire software development lifecycle, encompassing requirement gathering, design, implementation, testing, and deployment, is strictly

bounded by the academic timeline mandated by the Software Engineering Teaching Unit of the University of Kelaniya.

- **Team Capacity:** All system engineering and development tasks must be exclusively executed by the four assigned members of Team ZYNAPTRIX. Third-party development outsourcing is strictly prohibited.
- **Content Creation:** The generation and formatting of all educational materials (videos, notes, and quizzes) are strictly designated as the responsibility of the client; content creation is not accommodated within the development timeline.

CHAPTER 12

ASSUMPTIONS

The fundamental assumptions regarding the operational environment, user capabilities, and external technological dependencies upon which the successful deployment and operation of the EDURA Learning Management System rely are detailed in this section. If any of these foundational assumptions are proven to be incorrect, the project timeline, scope, or system functionality may be significantly impacted.

12.1 User Capabilities and Access

- **Digital Literacy:** It is assumed that a foundational level of computer literacy is possessed by all target users, including students, teaching staff, and administrators, enabling them to navigate standard web-based applications.
- **Device and Network Availability:** It is assumed that reliable access to personal computers, laptops, or suitable mobile devices, accompanied by a stable broadband internet connection, is consistently maintained by the students and staff.

12.2 External Systems and Dependencies

- **Third-Party Service Uptime:** It is assumed that the external services integrated into the platform, specifically the Resend email API used for transactional email, Google's sign-in service, and Neon, will remain consistently operational and highly available.
- **Browser Usage:** It is assumed that modern, updated web browsers (such as Google Chrome, Mozilla Firefox, or Microsoft Edge) are utilized by the end-users to access the platform, as backward compatibility with deprecated browsers (e.g., Internet Explorer) is not accommodated.
- **Hosting Environment:** It is assumed that an adequate cloud hosting environment and database infrastructure will be provisioned by the client to support the storage requirements of the uploaded video lectures and educational content.
- **Cloud Platform Infrastructure:** It is assumed that Microsoft Azure will provide reliable cloud infrastructure services, including compute resources, networking, and storage capabilities necessary for containerized microservice deployment. It is

further assumed that Azure's service-level agreements (SLAs) will meet the system's uptime and performance requirements.

- **Payment Gateway Uptime:** It is assumed that high availability and consistent operational uptime will be maintained by the external PayHere API to ensure the uninterrupted processing of automated student subscription payments.
- **Media Platform Uptime:** It is assumed that high availability and uninterrupted streaming services will be consistently maintained by YouTube and Cloudinary.

12.3 Operational and Business Assumptions

- **Content Provisioning:** It is assumed that all necessary educational materials, including recorded video lectures, structured notes, and formatted quiz questions, will be completely prepared and supplied by the client (Mr. Dimuthu Prabath) prior to the system's final deployment phase.
- **Administrative Responsiveness:** Because subscription activations are reliant on manual verification, it is assumed that uploaded payment receipts will be reviewed and processed by the system administrators in a timely manner to ensure that student access is not unreasonably delayed.
- **Academic Integrity:** While anti-cheating mechanisms are implemented within the system, it is assumed that a general standard of academic honesty is maintained by the students during unsupervised online assessments.
- **Client Account Provisioning Readiness:** It is assumed that all necessary third-party administrative and financial accounts, specifically a verified PayHere merchant account, a dedicated private YouTube channel, and a configured Cloudinary environment, will be provisioned and possessed by the client (Mr. Dimuthu Prabath) prior to the system's final deployment phase. At the final release the PayHere integration still uses PayHere's sandbox; enabling it requires the live merchant account. It is further assumed that the embedding of private videos strictly complies with YouTube's prevailing terms of service for educational platforms.

CHAPTER 13

USE CASE DIAGRAMS

The functional requirements and the interactions between the external actors and the EDURA Learning Management System are visually represented within this section. The behavioral perspective of the system is illustrated through these diagrams, detailing how the distinct user roles interact with the core functionalities of the platform.

13.1 System Actors

Three primary actors by whom the system is utilized have been identified:

- **Student:** Engagement with educational materials, submission of assessments, uploading of payment receipts, and tracking of personal academic progress are performed by this actor.
- **Teacher:** The creation of course modules, uploading of video lessons, configuration of interactive quizzes, and monitoring of class performance are executed by this actor.
- **Administrator:** The overarching management of the platform, including user role assignment, system configuration, and the manual verification of uploaded subscription receipts, is overseen by this actor.

13.2 Student Subsystem Use Case Diagram

The specific functionalities accessible to the student actor, as implemented, are detailed in this subsystem diagram: registration with email verification, login (with optional Google sign-in and password reset), browsing courses and class packages, enrolment and payment by uploading a bank-deposit receipt, watching video lessons with checkpoint questions, taking timed assessments under anti-cheat monitoring, and viewing results, leaderboards and badges, progress and feedback, and the profile. The PayHere payment is implemented in the backend but hidden in the checkout page, and a certificate download is not provided. Figures 2 and 3 are the diagrams approved in version 1.0.

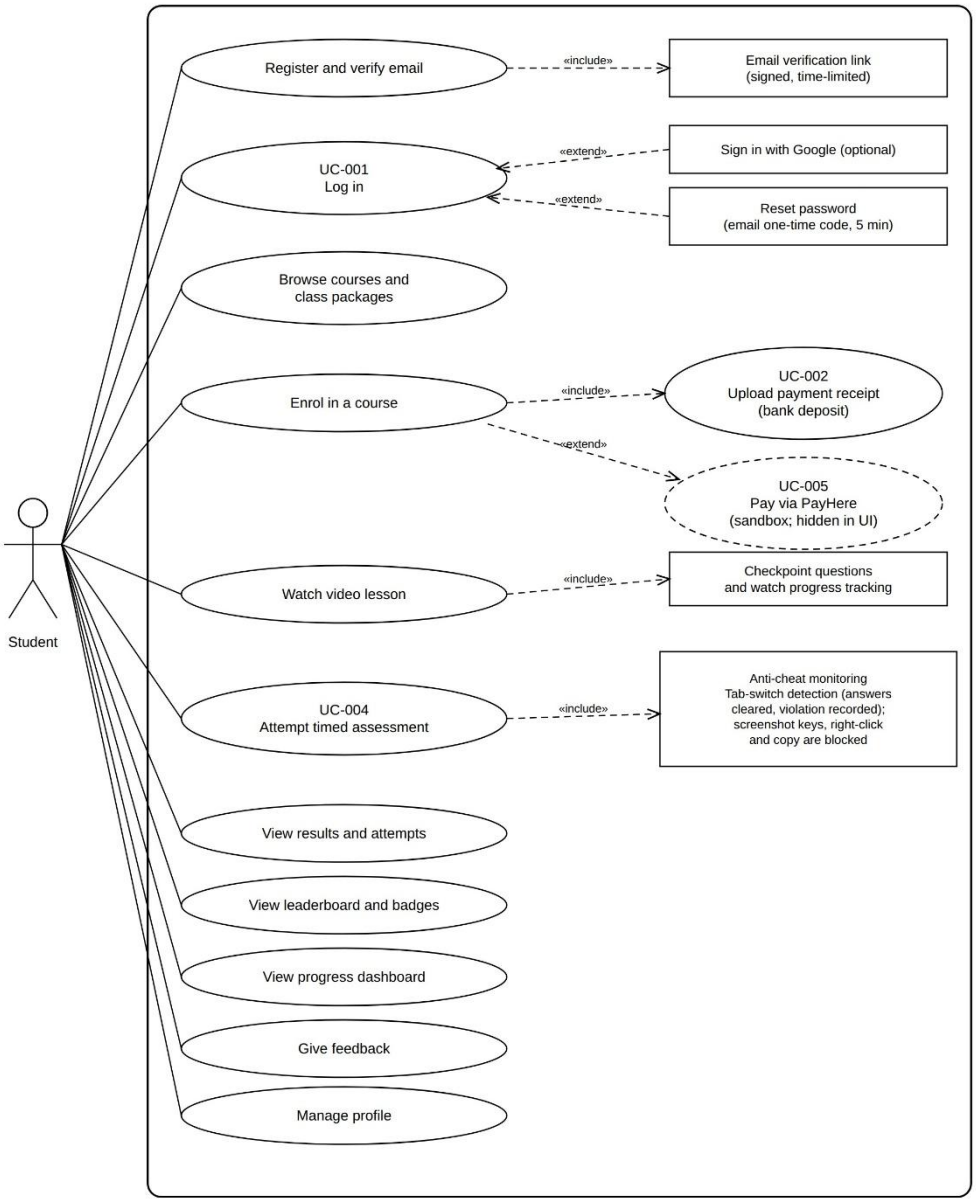


Figure 1: Student Subsystem Use Case Diagram

13.3 Teacher Subsystem Use Case Diagram

The instructional and content management capabilities provided to the educational staff are mapped in the following diagram. The processes by which lessons are structured, multiple-choice questions (MCQs) are authored, and student analytical reports are generated are demonstrated.

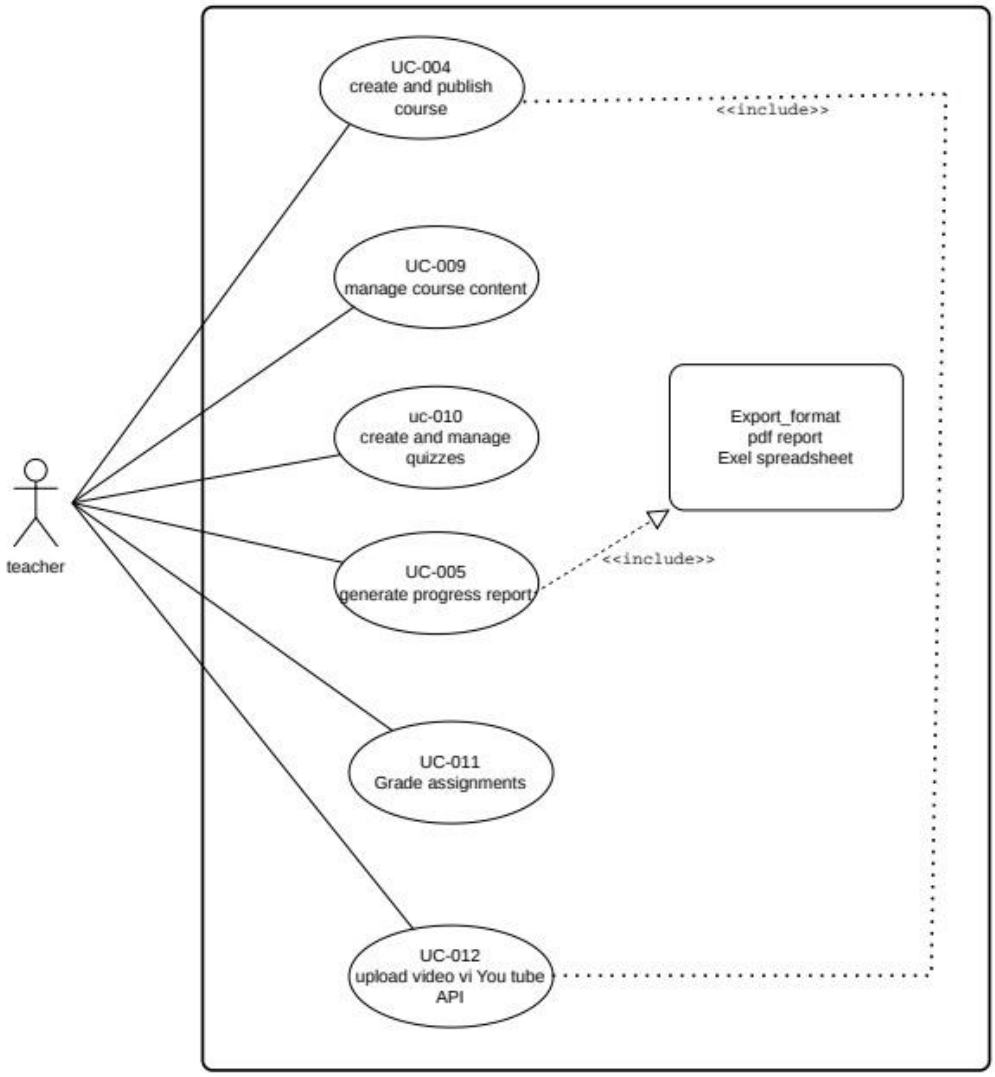


Figure 2: Teacher Subsystem Use Case Diagram

13.4 Administrator Subsystem Use Case Diagram

The operational and administrative tasks required to maintain the platform are isolated in this final diagram. The workflows through which user accounts are managed and manual subscription payments are verified and approved, are outlined.

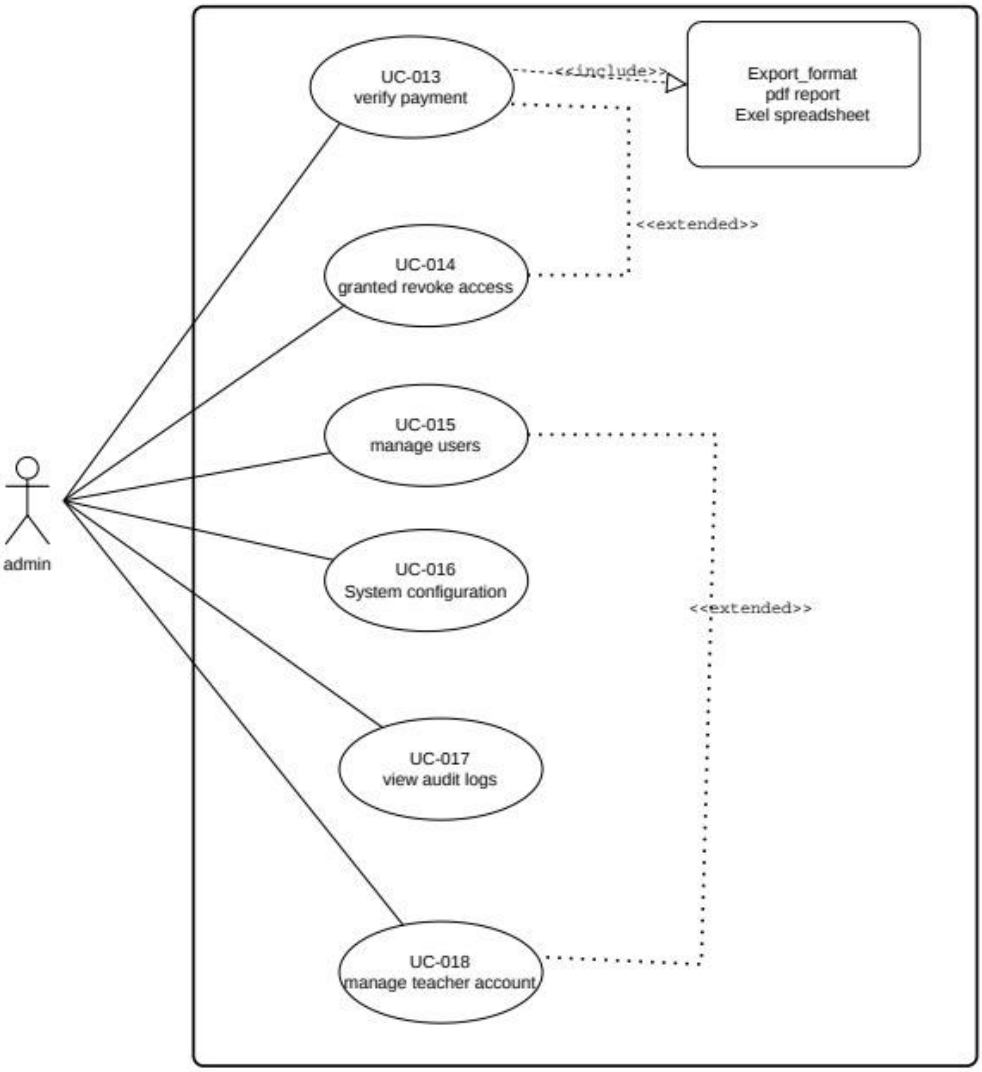


Figure 3: Administrator Subsystem Use Case Diagram

CHAPTER 14

USE CASE DESCRIPTIONS

The step-by-step main success scenarios, alternative flows, and system states for the critical system use cases are documented within this section. The precise sequence of interactions between the identified actors and the EDURA Learning Management System is detailed to provide a comprehensive behavioral blueprint for the development team.

14.1 UC-001: User Authentication and Login

Table 4: UC-001 – User Authentication and Login

Element	Description
Use Case ID & Name	UC-001: User Authentication and Login
Primary Actor	Student, Teacher, Administrator
Preconditions	A valid account must have been previously registered and its email address verified by the user.
Main Success Scenario	1. The login page is accessed by the user. 2. The email address and password are entered and submitted by the user. 3. The credentials are validated by the system, and it is confirmed that the email address has been verified. 4. A short-lived access token (15 minutes) is issued to the user, and a refresh token (7 days) is stored in an HttpOnly cookie; a Redis-backed session is created (at most three active sessions per user). 5. The user is redirected to the appropriate role-based dashboard by the system.
Alternative Flows	3a. Invalid Credentials: An error message is displayed by the system, and the user is prompted to re-enter the credentials.

	3b. Email Not Verified: The user is informed that the email address must be verified, and a new verification message can be requested. 2a. Google Sign-In: The user selects Google sign-in; the frontend obtains a Google access token, the system verifies it with Google, links it to the existing account by email address (or creates the account), and continues from step 4. 5a. Expired Access Token: The refresh token is used to obtain a new access token; refresh tokens are rotated on each use. Password recovery and email verification use one-time passwords with a 5-minute lifetime; an invalid or expired code produces an error message and a new code can be requested.
Postconditions	An active, authenticated session is established for the user, granting role-specific access to system resources.

14.2 UC-002: Upload Subscription Receipt

Table 5: UC-002 – Upload Subscription Receipt

Element	**Description**
Use Case ID & Name	UC-002: Upload Subscription Receipt
Primary Actor	Student
Preconditions	The student must be authenticated and logged into the platform. A digital copy of the payment receipt must be possessed by the student.
Main Success Scenario	1. The subscription portal is navigated to by the student. 2. The digital receipt file (JPEG, PNG or PDF) is selected and uploaded by the student.

	3. The relevant payment details (e.g., course and amount) are entered and submitted by the student. 4. The file type and content are validated, the file is stored in Cloudinary, and the payment is recorded as pending with a receipt awaiting review by the system. 5. A confirmation notification is displayed to the student, and a receipt-uploaded event is published.
Alternative Flows	**2a. Invalid File Format/Size:** The upload is rejected by the system, and a warning message specifying the accepted formats and size limits is displayed.
Postconditions	The receipt location is stored with the payment record and flagged for manual administrative review; approval grants the course enrollment.

14.3 UC-003: Create Educational Module

Table 6: UC-003 – Create Educational Module

Element	Description
Use Case ID & Name	UC-003: Create Educational Module
Primary Actor	Teacher
Preconditions	The teacher must be authenticated. Sufficient storage capacity must be available on the cloud server.
Main Success Scenario	1. The course management screens in the admin panel are accessed by the teacher. 2. The option to create a new module is selected. 3. The module title, description, and required parameters are input by the teacher. 4. The private YouTube video ID/URL and any associated Cloudinary media links are entered, and supplementary PDF notes are uploaded. 5. The external media links and uploaded files are validated and the references are stored in the database by the system. 6. The course is published (draft to published), and the new educational module is rendered accessible to enrolled students.
Alternative Flows	**5a. Upload Interruption:** An error is detected by the system, the incomplete file

	is discarded, and the teacher is prompted to restart the upload process.
Postconditions	A new, structured educational module has been successfully added to the system database and made visible to authorized students.

14.4 UC-004: Attempt Timed Assessment

Table 7: UC-004 – Attempt Timed Assessment

Element	**Description**
Use Case ID & Name	UC-004: Attempt Timed Assessment
Primary Actor	Student
Preconditions	The student must be authenticated and enrolled in the course with an active subscription. The assessment must be actively published by the teacher.
Main Success Scenario	1. The specific assessment is opened by the student. 2. The start option is initiated. 3. A timed session is created in Redis and the countdown timer is commenced; tab-switch and focus-loss monitoring is activated by the system. 4. The answers (MCQ/True-False/short answer) are selected or entered, and the final submission is executed by the student.

	5. The objective questions are automatically graded by the system; short-answer questions are held for manual review. 6. The score is recorded in the database, and the results are displayed to the student.
Alternative Flows	3a. Tab Switching or Focus Loss Detected: The violation is recorded and reported to the administrators and relevant teaching staff. 4a. Time Expiration: The timer reaches zero, and the assessment is automatically submitted by the system utilizing the currently selected answers.
Postconditions	The student's grade is permanently recorded within the database, and the overall course progress metrics are updated by the system.

14.5 UC-005: Automated Subscription Payment via PayHere

Table 8: UC-005 – Automated Subscription Payment via PayHere

Element	**Description**
Use Case ID & Name	UC-005: Automated Subscription Payment via PayHere
Primary Actor	Student
Secondary Actor	PayHere Payment Gateway (External System)
Preconditions	The student must be authenticated. A PayHere merchant account must be configured; the delivered system uses PayHere's sandbox, and the option is hidden in the checkout page.

Main Success Scenario	1. The subscription portal is accessed by the student. 2. The PayHere payment option is selected. 3. The student is redirected to the external PayHere checkout gateway by the system. 4. Valid payment details are entered and submitted by the student on the external gateway. 5. The transaction is authorized and processed by PayHere. 6. A payment status webhook (server-to-server callback) is transmitted by PayHere; its signature is verified and the payment is recorded once, keyed on the transaction identifier. 7. A payment-success event is published and the Enrollment Service activates the student's enrollment. 8. The student is redirected back to the EDURA dashboard, and a payment confirmation message is displayed.
Alternative Flows	4a. Payment Declined: A failed status is returned by the gateway. The student is redirected back to the EDURA payment page, an error message is displayed, and course access is not granted.

	6a. Webhook Timeout / Network Failure: The webhook is not received. The subscription is not activated; a reconciliation polling process was not implemented, so the student can use the bank-deposit flow or an administrator resolves the payment. 3a. User Cancellation: The payment process is canceled by the user while on the external gateway. The user is redirected back to the EDURA portal without any changes being made to the subscription status.
Postconditions	The subscription fee is processed, and automated access to the course materials is granted to the student without manual administrative intervention. Implementation status: backend implemented and tested against a mock gateway; hidden in the user interface (LIM-10).

CHAPTER 15

ACTIVITY DIAGRAMS

The dynamic behavioral aspects of the EDURA Learning Management System are depicted within this section. The sequential flow of activities, control flows, and decision-making pathways for critical system processes is graphically illustrated through activity diagrams. A clear understanding of the step-by-step execution of core platform functionalities is provided by these models.

15.1 User Authentication and Verification Flow

The sequence of operations executed during the user registration, verification and login process is detailed in this diagram. The pathways for credential validation, role identification and email verification are outlined. In the implemented system a one-time password is not requested at every login: login issues access and refresh tokens directly, while one-time passwords are used for email verification and password recovery. Optional Google sign-in is an additional path. The diagram in this chapter is the one approved in version 1.0.

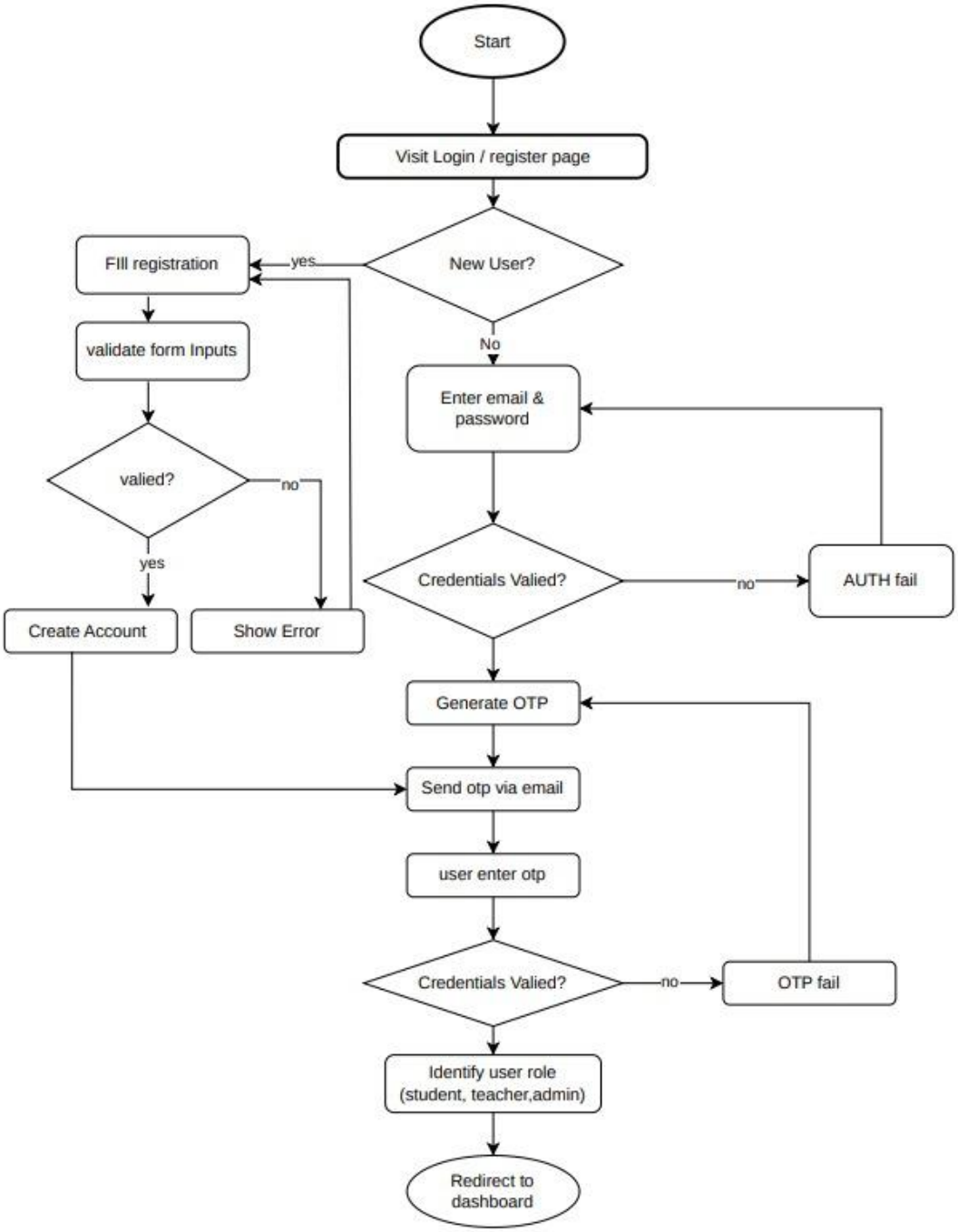


Figure 4: User Authentication and Verification Activity Diagram

15.2 Lesson and Content Management Flow

The step-by-step procedure by which new educational modules are created by the teaching staff is mapped in the following diagram. The sequential actions required for validating and embedding private YouTube video links, integrating Cloudinary media assets, attaching supplementary notes, and finalizing course structures are demonstrated.

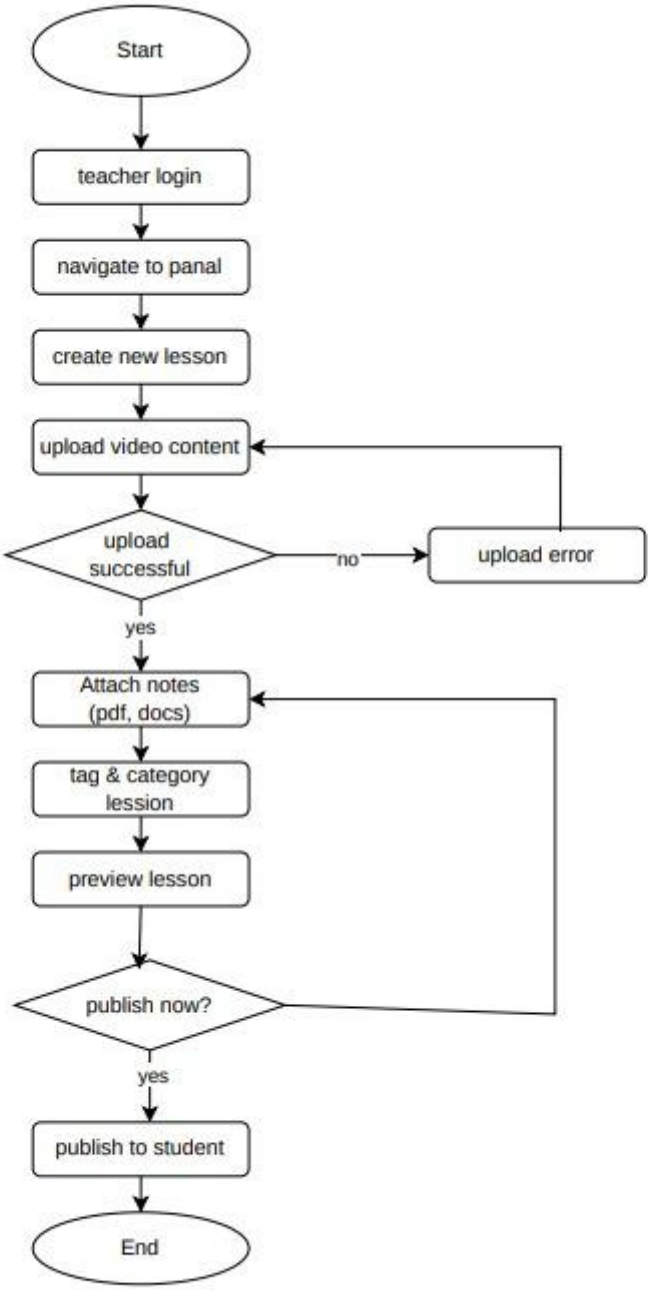


Figure 5: Lesson and Content Management Activity Diagram

15.3 Assessment and Auto-Grading Flow

The sequence of events occurring when a formal quiz is attempted by a student is represented in this diagram. The initiation of timed sessions, the enforcement of anti-cheating mechanisms, the submission of answers, and the subsequent automated calculation and recording of final grades are explicitly illustrated.

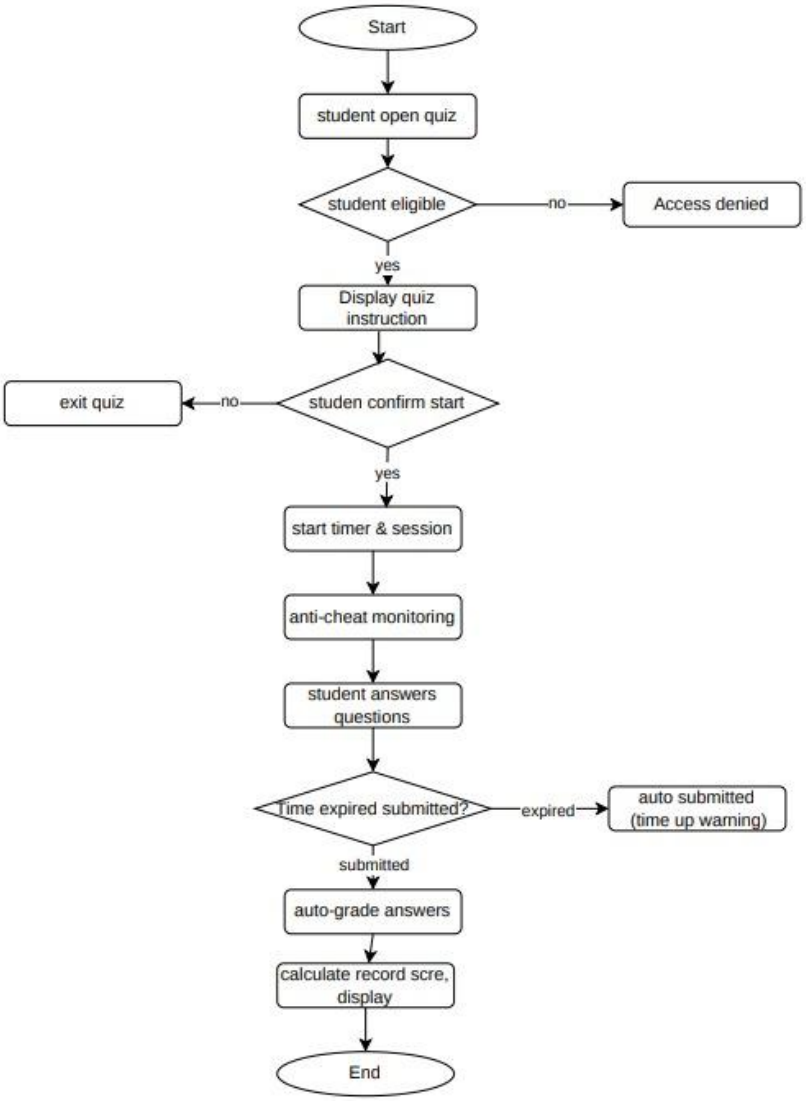


Figure 6: Assessment and Auto-Grading Activity Diagram

15.4 Subscription Payment Verification Flow

The administrative workflow associated with the processing of student subscription payments is outlined in this final diagram. The uploading of manual payment receipts by the student, followed by the visual review, verification, and subsequent approval or rejection by the system administrator, is systematically detailed.

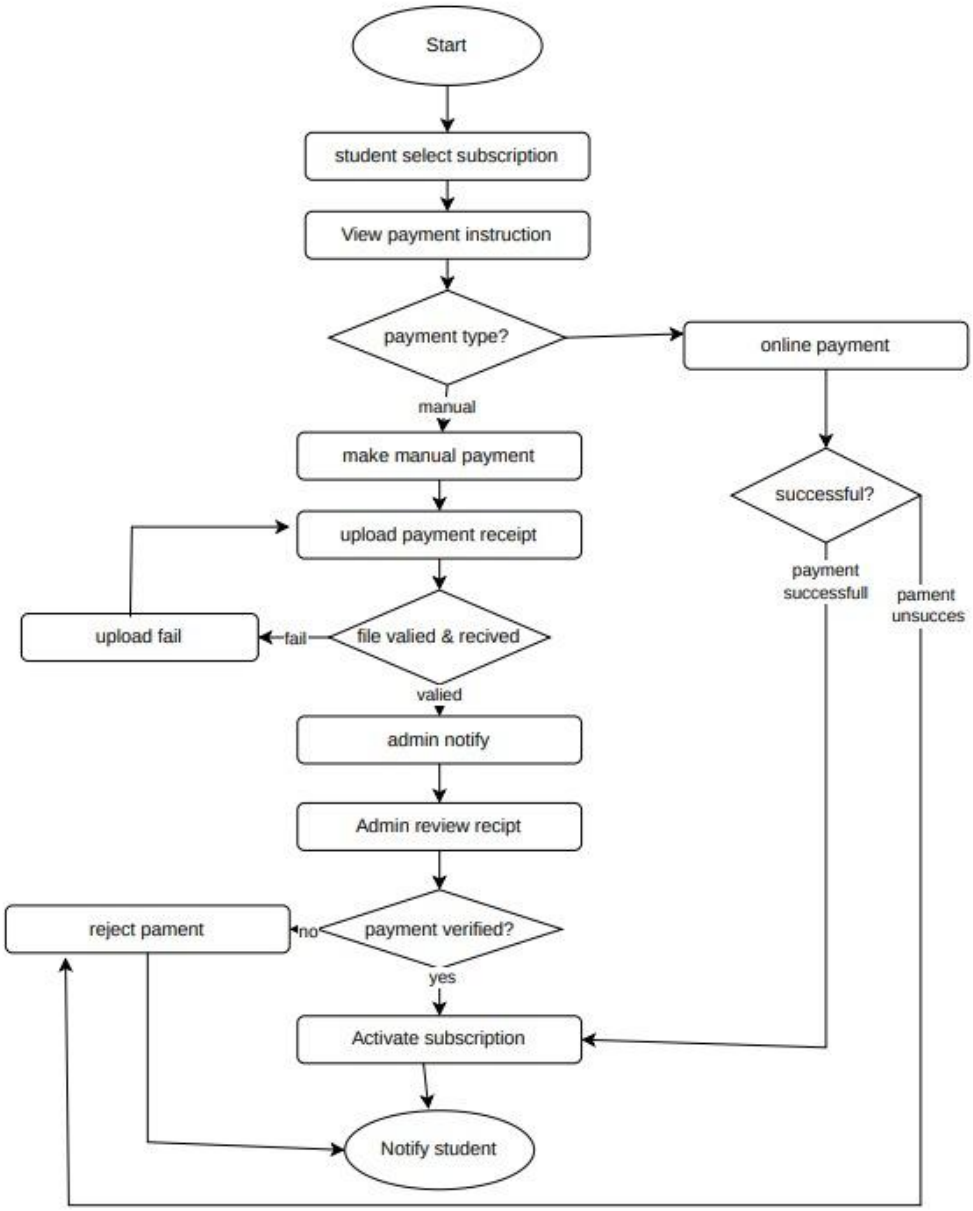


Figure 7: Subscription Payment Verification Activity Diagram

CHAPTER 16

EXTERNAL INTERFACE REQUIREMENTS

The external interface requirements for the EDURA Learning Management System are formally specified in this section. A seamless and secure interaction between the software system, its end-users, underlying hardware infrastructures, and external third-party services is ensured through these specifications.

16.1 User Interfaces

The user interface (UI) is required to be fully responsive, ensuring that optimal viewing and interaction experiences are provided across a wide spectrum of devices, including desktop monitors, tablets, and mobile phones. The visual design is mandated to be intuitive, clean, and aligned with standard educational platform aesthetics.

Distinct interface layouts and dashboards must be presented based on the authenticated user's role:

- **Student Interface:** A centralized dashboard by which enrolled courses, active assessments, gamification leaderboards, and overall progress metrics are accessed. A checkout page for the bank-deposit receipt upload is provided; the PayHere option is hidden in the delivered release.
- **Teacher Interface:** Course, module, lesson and assessment builders through which video lessons and notes are added and MCQs are authored. These screens are delivered within the admin panel; a separate teacher route does not exist. Student progress can be viewed and PDF progress reports are generated.
- **Administrator Interface:** A system management console utilized for the moderation of user accounts, the review of student payment receipts, the review of violations, feedback and support chat, and analytics.

16.2 Hardware Interfaces

As a strictly web-based platform, no bespoke hardware components are directly interacted with by the EDURA application on the client side. However, the following hardware requirements are assumed for optimal system operation:

- **Client-Side Hardware:** Standard web-enabled devices (e.g., desktop computers, laptops, tablets, or smartphones) equipped with functional displays and standard

input devices (keyboards/mice/touchscreens) are required to be utilized by the end-users.

- **Server-Side Hardware:** The application is expected to be hosted on standard cloud infrastructure. Hardware interfaces at the server level (such as CPU allocation, RAM, and physical storage for video content) will be abstracted by the chosen cloud hosting provider and managed via standard operating system protocols.

16.3 Software Interfaces

To facilitate extended functionalities, seamless communication with several external software components and application programming interfaces (APIs) is required by the EDURA system.

- **Google Gemini AI API:** This interface was not implemented. The planned digital learning assistant was descoped (FR-011, LIM-01).
- **Resend Email API:** Instead of an SMTP server, the Resend transactional email API is used by the Notification Service for verification and password-reset messages, payment confirmations, violation notices and progress reports, because SMTP configuration proved unreliable in the CI, staging and local environments.
- **PostgreSQL Database:** PostgreSQL is hosted on Neon, with one database per service (for example auth_db, course_db and payment_db), accessed through the SQLAlchemy ORM with Alembic migrations. It stores user profiles, course structures, lesson metadata, assessment records, enrollments and payment transaction data. This replaces the MongoDB database of version 1.0.
- **Redis Cache:** Redis is used for one-time passwords with automatic expiry, sessions, timed assessment sessions, leaderboard sorted sets and cached summary analytics.
- **PayHere REST API / Webhook Interface:** The payment gateway interface is implemented in the backend. Payment sessions are initialised by transmitting signed request payloads, and asynchronous server-to-server confirmations (webhooks) are authenticated by signature verification and processed idempotently to update the payment and enrollment status. The integration uses PayHere's sandbox and is hidden in the checkout page of the delivered release.

- **API Gateway:** An Nginx reverse proxy is the single entry point for browser requests. It routes each /api/<service>/ path to the corresponding microservice, limits requests to 100 per minute per client address (burst of 20), sets CORS headers and limits request bodies to 25 MB. In the cloud deployment a Caddy edge proxy terminates HTTPS with automatically renewed certificates.
- **Microsoft Azure Cloud Services:** The services run on Azure Kubernetes Service in a staging namespace, with images built and deployed by GitHub Actions pipelines. Payment receipts and supplementary notes are stored in Cloudinary rather than in Azure Blob Storage, and Azure Monitor integration was not implemented; service health is checked by the pipeline and by readiness and liveness probes.
- **Inter-Service Communication:** Services do not call one another over HTTP. They exchange asynchronous events through a RabbitMQ topic exchange (edura.events) with durable queues and a dead-letter exchange, and keep small read-only replicas of the data they need.
- **YouTube IFrame Player API:** Private lecture videos are embedded through the privacy-enhanced youtube-nocookie.com player within the student interface.
- **Cloudinary API:** Cloudinary is used for signed delivery of supplementary media (signed URLs expiring after one hour) and for the storage of payment receipts and notes.
- **Google Sign-In:** Optional sign-in with Google: the frontend obtains a Google access token, which the Auth Service verifies with Google before issuing its own tokens.

16.4 Communication Interfaces

The protocols and standards by which data is transmitted between the client browsers, the web server, and external APIs are explicitly defined as follows:

- **HTTPS Protocol:** All external traffic is encrypted using HTTPS; the Caddy edge proxy terminates TLS with automatically renewed Let's Encrypt certificates, and HSTS and other security headers are set. This prevents the interception of sensitive user credentials and assessment data.

- **RESTful Architecture:** Communication between the front-end user interface and the back-end server is required to be structured according to REST (Representational State Transfer) principles.
- **Data Formatting:** The JSON (JavaScript Object Notation) format shall be exclusively utilized for the packaging and transmission of data payloads between the server and the web client.

CHAPTER 17

DATA REQUIREMENTS

The logical structuring, storage mechanisms, and security protocols concerning the information managed by the EDURA Learning Management System are outlined in this section. The persistent and secure storage of user profiles, educational content, financial records, and assessment grades is ensured through these specifications.

17.1 Logical Data Entities

The underlying conceptual structure of the system data is defined by several core entities. The interrelations between these entities are strictly configured to maintain referential integrity across the system.

- **User Entity:** Authentication credentials (bcrypt-hashed passwords), role designations (Admin, Teacher, Student), and profile information are stored by the Auth and User services; extended profiles, streams, grade subjects and referral records are stored by the User Service.
- **Course and Lesson Entities:** The hierarchical organization of educational modules is managed. Instead of video file paths, the youtube_video_id, cloudinary_asset_url and cloudinary_public_id are stored alongside lesson metadata, material records and video checkpoint questions, with a draft, published and archived lifecycle.
- **Assessment Entity:** The structural configurations for quizzes, including question sets, predefined correct answers, and allocated time limits, are recorded by the Assessment Service, together with badges.
- **Submission, Grading and Violation Entities:** Student assessment attempts, specifically selected answers, automatically calculated final scores, completion timestamps and recorded violations, are captured.
- **Payment and Receipt Entities:** Financial records are maintained in the Payment Service. The payment_method (an enumeration of MANUAL or PAYHERE), the external transaction_id (retrieved from the payment gateway), and the payment_status (an enumeration of PENDING, SUCCESS or FAILED) are recorded; manual receipts are stored in a separate receipt record (status PENDING, APPROVED or REJECTED) that holds the receipt location. Bank account details are stored for the deposit flow.

- **Enrollment, Progress and Administration Entities:** Enrollments (active, suspended, with an expiry date) are owned by the Enrollment Service; progress records, certificates and leaderboard data by the Progress Service; and the audit log, feedback and support chat by the Admin Service.

17.2 Data Storage Mechanisms

PostgreSQL, hosted on Neon, is used for the structured storage of all educational content, user profiles, assessment data, and transactional records, with one database per service and no service reading another service's database. The schema is managed with Alembic migrations. This replaces the MongoDB document store mandated by version 1.0, because relational integrity and ACID guarantees were required for payments, enrollments and grades.

Redis, an in-memory data structure store, is deployed alongside PostgreSQL to provide:

- Session state management for authenticated users
- Temporary storage of One-Time Passwords (OTPs) with automatic expiration (five minutes)
- Timed assessment sessions, cached leaderboard data and aggregated analytics
- Session state and refresh tokens for revocation

External media references (YouTube video IDs and Cloudinary asset URLs) are stored as string fields in the database, eliminating the need to store video files. Supplementary notes and image-based payment receipts are stored in Cloudinary (not in Azure Blob Storage); only their URL and public identifier are kept in the database.

17.3 Data Security and Integrity

Strict data protection mechanisms are required to be enforced at the database level to prevent unauthorized access and data breaches.

- **Cryptographic Hashing:** All user passwords must be securely hashed utilizing modern cryptographic algorithms (e.g., bcrypt) prior to storage; the database storage of plain-text passwords is strictly prohibited.

- **Access Control:** Direct database access is severely restricted. Modifications and queries are permitted to be executed exclusively through authorized back-end API services or by designated high-level database administrators.
- **Input Validation:** Strict data typing is applied to all incoming payloads through Pydantic schemas, database access uses SQLAlchemy parameterized queries, and uploaded files are limited by size, type and file signature, preserving data consistency and preventing injection vulnerabilities.

17.4 Data Retention and Backup

Version 1.0 required automated daily backups retained for thirty days in a geographically distinct location. No scheduled backup job was implemented in the project; data protection relies on the managed database provider, and the backup and retention behaviour has not been verified (NFR-003).

CHAPTER 18

ALTERNATIVE SOLUTIONS CONSIDERED

The various architectural approaches, platform delivery methods, and technical methodologies that were evaluated during the initial design phase of the EDURA Learning Management System are detailed in this section. Furthermore, the justifications for the ultimately selected solutions are provided to demonstrate the rationale behind the system's design.

18.1 Architectural Approach: Monolithic vs. Decoupled RESTful API

During the initial planning phase, traditional monolithic architecture was considered due to its perceived simplicity in early-stage development and deployment. However, this approach was ultimately rejected because flexibility is significantly limited by monolithic structures, and future integrations or system scaling are often complicated by tightly coupled codebases.

To implement this microservice architecture, the system uses a modern technology stack. The frontend user interface is built using the Next.js framework with TypeScript. Backend microservices (eleven in total) are implemented in Python with FastAPI. All services are containerized using Docker and deployed on Microsoft Azure Kubernetes Service, with PostgreSQL (Neon) as the primary database, Redis providing session management and caching, and RabbitMQ carrying asynchronous events between the services.

The adoption of microservices architecture in educational platforms has been extensively validated in recent literature. Studies demonstrate that microservices enable independent scalability of system components, reduce deployment time by up to 40%, and significantly improve fault isolation compared to monolithic architectures [14][15][16]. In e-learning contexts specifically, microservices-based LMS implementations have achieved infrastructure cost reductions of 50-70% while maintaining superior performance and availability [17][18].

18.2 Platform Delivery: Native Mobile Application vs. Responsive Web Application

The development of dedicated native mobile applications (specifically for iOS and Android platforms) was extensively evaluated to maximize mobile user engagement. However, this alternative was dismissed due to the strict academic time constraints imposed on the project and the limited capacity of the four-member development team. The simultaneous

maintenance of multiple codebases was deemed unfeasible within the allocated project lifecycle.

Therefore, a fully responsive web-based application was chosen as the primary delivery mechanism. This approach is justified because cross-platform compatibility across desktops, tablets, and smartphones is guaranteed by responsive web design without incurring the overhead of native app development.

18.3 Payment Processing: Automated Gateway vs. Manual Verification vs. Hybrid Approach

To process student subscriptions, the exclusive use of a fully automated payment gateway and the exclusive use of manual receipt verifications were initially evaluated as mutually exclusive alternatives. However, a hybrid approach to subscription processing was ultimately selected as the optimal solution.

To automate transaction processing and reduce the administrative overhead associated with visual verifications, the PayHere payment gateway was integrated into the backend. Concurrently, the manual receipt upload functionality was retained as a fallback mechanism and, because the PayHere option is hidden in the delivered checkout page, it is the active payment method. This dual-method approach is justified because an equitable and accessible system is ensured for all students, particularly those who do not have bank accounts, credit cards, or debit cards, while the workflow for system administrators remains manageable.

18.4 Content Delivery: Local Server Storage vs. Third-Party Media Platforms

The storage and streaming of large binary files, particularly extensive pre-recorded video lectures, were initially proposed to be handled directly on the local web server's file system. This method was rejected because server bandwidth and storage capacity are rapidly exhausted by heavy media files, which invariably leads to severe performance degradation during concurrent student access.

Alternatively, a specialized external media hosting strategy was selected. Primary video lectures are mandated to be uploaded to a private YouTube channel, ensuring that video hosting is offloaded to a highly scalable network and access is strictly restricted to website redirection and embedding. Furthermore, Cloudinary is utilized to manage advanced video streaming capabilities, media quality optimization, and additional asset delivery. This decision is justified because high availability, adaptive bitrate streaming, and the complete

preservation of local server bandwidth are inherently provided by these dedicated platforms, ensuring that system performance is not compromised by large media distributions.

18.5 Database: MongoDB vs. PostgreSQL

Version 1.0 mandated a MongoDB document store and excluded relational databases. During design this was reversed: payments, enrollments and grades need relational integrity and ACID transactions, and the SQLAlchemy and Alembic ecosystem supports schema migrations for each service. PostgreSQL on Neon was therefore selected, with one database per service; Redis was retained for caching and short-lived state.

18.6 Email Delivery: SMTP Server vs. Transactional Email API

An SMTP server was planned for one-time passwords and notification messages. SMTP configuration was unreliable in the CI, staging and local environments, so the Resend transactional email API was adopted.

18.7 Inter-Service Communication: Synchronous REST vs. Asynchronous Events

Version 1.0 allowed either internal REST calls or a message broker. Synchronous calls coupled the availability of services (for example, content delivery would fail whenever the Enrollment Service was down), so the services communicate only through events on a RabbitMQ topic exchange and keep small read-only replicas of the data they need.

CHAPTER 19

FEASIBILITY STUDY

A comprehensive evaluation of the EDURA Learning Management System's viability across technical, economic, operational, and scheduling dimensions is presented in this section. The project's likelihood of success, given the established constraints and chosen technological integrations, is formally assessed.

19.1 Technical Feasibility

The project is determined to be technically feasible, although significant technical complexity is introduced by the chosen microservice architecture.

- **Architecture:** The implementation of independent microservices, an API gateway, and decentralized databases requires advanced containerization (e.g., Docker) and rigorous DevOps practices. However, fault isolation is substantially improved, and independent scaling is enabled by this approach, making it highly suitable for modern web applications.
- **Media Handling:** The severe technical risks associated with server bandwidth exhaustion and massive storage requirements are completely mitigated by the strategic offloading of video hosting to private YouTube channels and Cloudinary. High-quality, adaptive bitrate streaming is inherently guaranteed by these established third-party APIs.
- **Payment Integration:** The automated processing of subscriptions is facilitated by the PayHere API. The secure handling of webhooks was implemented and tested against a mock gateway; verification against the live gateway and the reconciliation of missed webhooks were not completed.
- **Technology Stack Maturity:** The chosen technology stack (Next.js, Python with FastAPI, PostgreSQL, Redis, RabbitMQ, Azure Kubernetes Service) represents mature, production-ready frameworks and platforms with extensive community support and documentation. The stack was implemented in full by the team and is covered by automated tests (backend coverage 98.6% of lines and 97.8% of branches; frontend 98.3% and 91.5%).

The original selection of MongoDB, supported by [19], was reversed during design in favour of PostgreSQL (Chapter 21). The combination of microservices with containerization

technologies like Docker and Kubernetes has demonstrated significant improvements in deployment efficiency and system resilience for e-learning platforms [20].

19.2 Economic Feasibility

The development and ongoing operation of the EDURA system are deemed highly economically feasible due to strategic architectural choices designed to minimize recurring costs.

- **Development Costs:** Because the software is being developed as an academic project by the four assigned members of Team ZYNAPTRIX, preliminary external development costs and third-party outsourcing fees are completely eliminated.
- **Operational and Hosting Costs:** Server infrastructure costs were minimized by the third-party media strategy (YouTube and Cloudinary), which avoids expensive video storage, by a free-tier or low-cost managed PostgreSQL database (Neon), and by a low-cost Kubernetes cluster for staging. The cluster was deprovisioned during part of development to conserve cloud credits. The consequence is limited capacity: about 20 concurrent clients per service were supported (NFR-009 not met).
- **Administrative Savings:** Because the PayHere option is hidden in the delivered release, the manual verification of payment receipts by administrators remains the working process; the saving expected from the automated gateway has not yet been realised.

19.3 Operational Feasibility

The system is evaluated as highly operationally feasible, as the workflows of all primary actors (Students, Teachers, and Administrators) are streamlined by the proposed solutions.

- **Student Accessibility:** The hybrid payment strategy ensures that students lacking digital banking capabilities are not excluded from the platform. In the delivered release the bank-deposit route is the active one.
- **Educator Workflow:** The lesson creation process is significantly expedited for the teaching staff. The time-consuming uploading of massive video files over potentially unstable internet connections is replaced by the rapid embedding of pre-existing YouTube and Cloudinary URLs.

- **Administrative Workflow:** Routine subscription activations are intended to be automated by the PayHere integration; until it is enabled, administrative effort is focused on the manual receipt review.

19.4 Schedule Feasibility

The delivery of the EDURA platform was bound by the academic timeline mandated by the Software Engineering Teaching Unit. The project was delivered, but the user-interface work and the integration tests were completed later than the sprint plan had scheduled, and several items were descoped (Chapter 21).

- **Parallel Development:** The simultaneous, non-blocking development of discrete system modules by the four team members is facilitated by the decoupled nature of the microservice architecture. For instance, the Authentication Service, Course Management Service, and Payment Service can be programmed and tested in parallel, thereby accelerating the overall development lifecycle.
- **Reduced Implementation Time:** The time previously allocated for the development of complex, custom video streaming algorithms and heavy file-handling logic is reclaimed through the utilization of YouTube and Cloudinary APIs, ensuring that the project deadlines can be confidently met.

CHAPTER 20

RISK ANALYSIS

Potential technical, operational, and external risks by which the successful deployment and continuous operation of the EDURA Learning Management System could be threatened are identified and evaluated in this section. To ensure system resilience and project success, proactive mitigation strategies for each identified vulnerability are formally defined.

20.1 Identified Risks and Mitigation Strategies

The primary project risks, categorized by their source and accompanied by their respective mitigation protocols, are detailed in the following table:

Table 9: Risk Analysis and Mitigation Strategies

Risk Category	Risk Description	Probability / Impact	Mitigation Strategy
Architectural	**Distributed Data Inconsistency:** Because the system is decoupled into independent microservices, network latency or partial transaction failures between services may be experienced, leading to inconsistent data states.	Medium / High	Robust inter-service communication protocols, circuit breaker patterns, and eventual consistency models (such as the Saga pattern) must be implemented. Comprehensive logging must be utilized to trace communication failures. **Outcome:** Implemented: services communicate through events with idempotent consumers and dead-letter handling; circuit breakers and the Saga

			pattern were not implemented.
External Dependency	**Third-Party Service Outages:** Critical system functionalities, specifically video streaming and automated payment processing, could be severely impacted if downtime is experienced by the YouTube, Cloudinary, or PayHere APIs.	Low / High	Comprehensive error handling and graceful UI degradation must be implemented. If payment webhooks fail, a background reconciliation process must be triggered to automatically poll the PayHere API for pending transaction statuses. **Outcome:** Partly implemented: error handling is in place; the reconciliation polling process was not implemented, and PayHere is hidden in the checkout page.
Security	**Unauthorized Content Distribution:** Private YouTube video links or Cloudinary asset URLs might be maliciously extracted from the source code and shared by active students with unauthorized individuals.	High / High	Domain-level embedding restrictions must be strictly enforced on the YouTube private channel (allowing playback only on the EDURA domain). Furthermore, dynamically generated, time-expiring signed

			URLs must be utilized for Cloudinary assets. **Outcome:** Partly implemented: Cloudinary signed URLs expire after one hour and YouTube is embedded in privacy-enhanced mode; the restriction of playback to the EDURA domain is a setting of the client's YouTube channel.
Operational	**Administrative Bottlenecks:** A significant backlog in subscription activations might be created if a large volume of manual payment receipts is uploaded by students simultaneously, particularly near examination periods.	Medium / Medium	The automated PayHere gateway must be heavily promoted to students as the primary payment method. Additionally, bulk verification tools must be provided within the administrator dashboard to expedite manual reviews. **Outcome:** Occurred: the PayHere option is hidden, so all payments are manual. Bulk verification tools were not delivered.
Technical	**Technology Stack Integration Complexity:** Integrating the Next.js frontend with	Medium / Medium	Establish clear API contracts using OpenAPI/Swagger specifications.

	eleven Python microservices, PostgreSQL databases, Redis caching, RabbitMQ and multiple third-party APIs (PayHere, YouTube, Cloudinary, Resend, Google sign-in) may introduce compatibility issues, serialization challenges, or deployment complexities.		Implement comprehensive integration testing in staging environments that mirror production Azure infrastructure. Utilize Docker Compose for local development to ensure environment parity. Maintain detailed technical documentation for each service interface. **Outcome:** Implemented: API contracts are generated as OpenAPI documents, integration and smoke tests run against staging, and Docker Compose is used locally.
Schedule	**Academic Deadline Overrun:** The project deadlines mandated by the university might not be met by the development team due to the inherent complexity of configuring microservices and integrating multiple external APIs.	Medium / High	Strict agile methodologies must be adhered to by Team ZYNAPTRIX. A Minimum Viable Product (MVP) encompassing only core functionalities must be prioritized and completed prior to the implementation of

			secondary features (e.g., gamification). **Outcome:** Occurred: the user interface and integration tests were completed later than planned and the AI assistant was descoped; the MVP scope was delivered.
Performance	Insufficient Capacity: The low-cost cluster cannot support the target of 500 concurrent users; a service stalled at about 40 concurrent clients.	High / Medium	Health probes and a second replica were added for the course service. The limitation is accepted for the release and recorded in ADR-17; the cluster must be scaled for production use.
Security	Known Hardening Gaps: no account lockout, authentication events not logged, a fallback signing key in the code, no Content-Security-Policy and no network policies.	Medium / Medium	Recorded in ADR-17 and the OWASP checklist as accepted limitations; security headers, a rate-limited gateway and dependency scanning are in place.

20.2 Continuous Risk Monitoring

The identified risks are not static; therefore, continuous monitoring throughout the software development lifecycle is required. Regular technical reviews and code audits must be conducted by the development team to ensure that the mitigation strategies outlined above are successfully integrated into the system's microservice architecture.

CHAPTER 21

CHANGES FROM VERSION 1.0

The differences between the requirements approved in version 1.0 and the system that was implemented are summarised in the following table. Each change arose from an implementation constraint, a client request or the time available.

Table 10: Changes from Version 1.0

Area	Version 1.0	Final system	Reason / reference
Database	MongoDB mandated; relational databases excluded.	PostgreSQL on Neon, one database per service; Redis for sessions, OTPs, timed assessments and leaderboards.	Relational integrity and ACID guarantees; SQLAlchemy and Alembic (ADR-02).
AI assistant (FR-011)	Google Gemini assistant.	Not implemented; support chat to the administration team.	API cost and time (LIM-01).
Email delivery	SMTP server.	Resend transactional email API.	SMTP unreliable in CI, staging and local environments.
Authentication (FR-001, FR-002, UC-001)	Password plus an OTP at every login; no social login.	Email and password login issuing JWT tokens; OTP codes for verification and password reset (5 minutes); optional Google sign-in.	Reduced login friction; convenience for students.
Anti-cheating (FR-013)	Browser lockdown and tab-switch detection.	Tab-switch and focus-loss detection, recorded and reported; no lockdown.	Behavioural monitoring only.
Payments (FR-007, FR-008)	PayHere as the primary method.	PayHere implemented in the backend on the sandbox and hidden in the checkout page; bank deposit with	Live merchant account not enabled (LIM-10).

		administrator verification is active.	
Payment reconciliation (NFR-004)	Polling of PayHere within 15 minutes.	Not implemented; webhook handling is idempotent.	Recorded as a limitation.
Reports (FR-012)	PDF and Excel exports.	PDF progress reports only.	Excel not delivered (LIM-07).
Certificates (FR-015)	Certificates generated and issued.	Eligibility rule, record and event implemented; no document or page.	Partly delivered (LIM-08).
Inter-service communication	REST or a message broker.	Asynchronous events only on a RabbitMQ topic exchange, with local read replicas.	REST calls coupled service availability (ADR-12).
Hosting and delivery	Azure with AKS or Container Instances; Azure Blob Storage and Azure Monitor.	Azure Kubernetes Service (staging) with GitHub Actions pipelines; Cloudinary for receipts and notes; no Azure Blob Storage or Azure Monitor.	Course guidelines and cloud credit limits (ADR-13).
Backups (NFR-003)	Daily backups kept for thirty days.	No project-level backup job.	Depends on the managed database provider; not verified.
Capacity (NFR-009)	500 concurrent users.	About 20 concurrent clients per service on the low-cost cluster.	Not met; accepted in ADR-17 (LIM-06).

Additional capabilities	Not in the SRS.	Class packages, streams and grade subjects, video checkpoint questions, feedback, support chat, audit log, referral and wallet ledger (FR-016 to FR-019).	Added to support class-based teaching and engagement.

Functional requirements FR-011 (AI assistant) and FR-012 (Excel export), and the non-functional requirements NFR-003, NFR-004 and NFR-009, are not satisfied by the final system; FR-007, FR-012 and FR-015 are partly satisfied. The known limitations are tracked as LIM-01 to LIM-10 in the final development report.

CHAPTER 22

REFERENCES

[1] Pan, Z., Biegley, L., Taylor, A., & Zheng, H. (2024). A Systematic Review of Learning Analytics: Incorporated Instructional Interventions on Learning Management Systems. *Journal of Learning Analytics*, 11(2), 52-72. https://doi.org/10.18608/jla.2023.8093

[2] Liliasari, L., & Riza, L. S. (2023). The Impact of Learning Management System (LMS) Usage on Students. *TEM Journal*, 12(2), 1082-1089. https://doi.org/10.18421/TEM122-54

[3] Simon, P. D., Jiang, J., & Fryer, L. K. (2024). Measurement of higher education students' and teachers' experiences in learning management systems: A scoping review. *Assessment & Evaluation in Higher Education*, 49(4), 441-452. https://doi.org/10.1080/02602938.2023.2266154

[4] Romdhoni, R. D., & Romdhoni, A. (2026). The Effectiveness of Learning Management Systems (LMS) in Enhancing Learning Experiences toward Achieving SDG 4: Quality Education. *Journal of Information System and Education Development*, 4(1), 39–46. https://doi.org/10.62386/jised.v4i1.210

[5] Aulianda, N., Wijayati, P. H., Ebner, M., & Schön, S. (2023). Analysis of Learning Management System towards Students' Cognitive Learning Outcome. *International*

Journal of Emerging Technologies in Learning (IJET), 18(23), 4–26. https://doi.org/10.3991/ijet.v18i23.36443

[6] Zeng, X. (2024). Exploring the impact of gamification on students' academic performance: A comprehensive meta-analysis of studies from the year 2008 to 2023. *British Journal of Educational Technology*, 55(4), 1471-1492. https://doi.org/10.1111/bjet.13471

[7] Nguyen-Viet, B., Nguyen-Duy, C., & Nguyen-Viet, B. (2024). How does gamification affect learning effectiveness? The mediating roles of engagement, satisfaction, and intrinsic motivation. *Interactive Learning Environments*, 33(3), 2635-2653. https://doi.org/10.1080/10494820.2024.2414356

[8] Zhang, L., & Wang, Y. (2024). Impact of Gamification on Students' Learning Outcomes and Academic Performance: A Longitudinal Study Comparing Online, Traditional, and Gamified Learning. *Education Sciences*, 14(4), 367. https://doi.org/10.3390/educsci14040367

[9] Alsabhan, W. (2023). Student Cheating Detection in Higher Education by Implementing Machine Learning and LSTM Techniques. *Sensors*, 23(8), 4149. https://doi.org/10.3390/s23084149

[10] Yulita, I.N., Hariz, F.A., Suryana, I., & Prabuwono, A.S. (2023). Educational Innovation Faced with COVID-19: Deep Learning for Online Exam Cheating Detection. *Education Sciences*, 13(2), 194. https://doi.org/10.3390/educsci13020194

[11] Hussein, F., Al-Ahmad, A., El-Salhi, S., Alshdaifat, E., & Al-Hami, M. (2022). Advances in Contextual Action Recognition: Automatic Cheating Detection Using Machine Learning Techniques. *Data*, 7(9), 122. https://doi.org/10.3390/DATA7090122

[12] Ozdamli, F., Aljarrah, A., Karagozlu, D., & Ababneh, M. (2022). Facial Recognition System to Detect Student Emotions and Cheating in Distance Learning. *Sustainability*, 14(19), 13230. https://doi.org/10.3390/su141913230

[13] Market Research Reports. (2024). Global Online Exam Proctoring Market 2023-2029. Cited in: HackerEarth Blog. (2025). Online Assessment Cheating Prevention Technologies. Retrieved February 2025 from https://www.hackerearth.com/blog/

[14] Sharma, S. (2025). The Impact of Microservices Architecture on System Scalability. *American Scientific Research Journal for Engineering, Technology, and Sciences*, 102(1), 140-148. https://asrjetsjournal.org/American_Scientific_Journal/article/view/11677

[15] Henríquez, C., Valencia, J. D. R., & Torres, G. S. (2025). Architectural Evolution at Netflix: A Case Study on Microservices and the Transformation from Monolithic to Scalable Systems. *Prospectiva*, 23(1), 1-14.

[16] Thallapally, N. (2024). Microservices architecture in cloud-native applications: Design patterns and scalability. *International Journal of Science and Research Archive*, 13(02), 4140-4145.

[17] Peng, C. F., et al. (2023). Design and Implementation of a Microservices-Based Online Learning Platform. In *Proceedings of the 2023 2nd International Conference on Educational Innovation and Multimedia Technology (EIMT 2023)* (pp. 455-460).

[18] eLearning Industry. (2022). Microservices Architecture And Migration in Learning Management Systems. Retrieved from https://elearningindustry.com/migrating-monolithic-learning-management-system-to-microservice-architecture

[19] Kumar, C. (2024). NoSQL Database Implementation for Learning Management Systems. *International Journal of Novel Research and Development*, 9(5), 793-801.

[20] Mynkis Digital Solutions. (2024). Microservices: Building Blocks for Scalable Educational Platforms. Retrieved from https://www.mynkis.com/articles/microservices-for-educational-platforms

CLIENT REQUIREMENT SIGN-OFF

The Software Requirements Specification (SRS) document for the EDURA Learning Management System has been thoroughly reviewed, and its contents are formally accepted by the undersigned.

It is acknowledged by all parties that the functional and non-functional requirements, business rules, structural constraints, and architectural decisions, as implemented and recorded in this final version, specifically encompassing the microservice architecture, the integration of third-party media platforms (YouTube and Cloudinary), and the hybrid payment mechanism (bank-deposit receipts with a PayHere integration), accurately reflect the delivered system and the changes listed in Chapter 21.

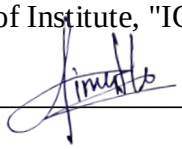
It is further understood that any subsequent modifications, additions, or removals to these formally agreed-upon requirements following the execution of this document must be subjected to a formal change control process and may impact the academic project timeline.

Accepted and Approved By:

1. Client / Project Sponsor

Name: Mr. Dimuthu Prabath

Title: Head of Institute, "ICT - Dimuthu"

Signature: 

Date: _____

2. Academic Supervisor (University of Kelaniya)

Name: _____

Title: _____

Signature: _____

Date: _____